Automated Testing Report for the Mini Project

BidBoard Auth Module

1. Project Overview

This mini project is an authentication and user management system built around the auth module. It handles user registration, login authentication, password strength checking, password reset/change, email verification, session persistence using browser storage, and admin validation. The main purpose of the project is to manage user access securely and consistently. User data is validated before being stored, and actions such as login, registration, password updates, and email verification depend on strict business rules. Important functions in this module include: - registerUser() - authenticateUser() - changePassword() - resetPassword() - verifyEmail() - calculatePasswordStrength() - generateVerificationCode() - saveUserSession() - getStoredUser() - getUsersFromStorage() These functions are central to the application’s workflow and must be validated carefully to prevent security issues and functional defects.

2. Testing Objectives

The objective of automated testing is to ensure that: - valid user operations work correctly - invalid inputs are rejected properly - authentication remains secure - email verification and password updates behave correctly - storage/session behavior is stable - important workflows continue to work after code changes The project needs three major kinds of testing: - Unit Testing - Integration Testing - API Testing These three layers together provide better confidence in the quality of the software and help reduce the risk of regression when updates are made.

3. Unit Testing

Unit testing checks a single function or method in isolation to ensure it behaves correctly for valid and invalid inputs. It is the first and most basic layer of quality assurance. Why this project needs unit testing: The logic in this project depends heavily on validation, password rules, user storage, and verification checks. A bug in one function can break registration, login, email verification, or password recovery. Unit test focus: - registerUser() - authenticateUser() - changePassword() - resetPassword() - verifyEmail() - calculatePasswordStrength() - generateVerificationCode() - saveUserSession() - getStoredUser() - getUsersFromStorage() Sample unit test cases: - valid registration with all required fields - duplicate email must be rejected - missing name, email, or password must fail - wrong password should fail - weak password must be rejected - short password should not be accepted - invalid verification code must fail - correct verification code must succeed - empty password should return a weak or none strength value - storage should

return empty values when no user data exists

4. Integration Testing

Integration testing verifies whether multiple modules or components work together correctly as a complete workflow. It checks the interaction between different functions and data flows. This project contains several important workflows that should be tested together: - Registration → Email Verification → Login - Registration → Password Change → Re-login - Password Reset → Login - Session persistence after reload - Error flow handling for invalid credentials Key integration scenarios: - a user registers successfully, verifies email, and then logs in - a user changes their password and cannot log in with the old password - a user resets the password and can log in with the new password - data is stored correctly in browser storage and retrieved accurately - invalid verification attempts do not modify the user record - failed login attempts do not create incorrect session state Integration testing ensures that the combined behavior is correct, even when the individual functions themselves pass unit tests.

5. API Testing

API testing checks whether backend endpoints behave correctly for valid and invalid requests. It focuses on request validation, authentication, status codes, and response messages. Even though this project currently uses front-end logic and local storage, these same tests are relevant if the project is later connected to a backend API. Possible endpoint coverage: - POST /api/auth/register - POST /api/auth/login - POST /api/auth/verify-email - POST /api/auth/change-password - POST /api/auth/reset-password - GET /api/users/me API test areas: - request validation - authentication and authorization - proper HTTP status codes - response body validation - error handling and negative cases Example API tests: - valid registration returns 201 - duplicate registration returns 409 - invalid login returns 401 - wrong email/password returns unauthorized error - correct verification request returns 200 - wrong verification code returns 400 - missing token in protected API returns 403 or 401 - empty request body returns bad request response

6. Test Strategy and Coverage

A proper automated test suite for this project should include both positive and negative tests: - valid input checks - invalid input checks - boundary conditions - empty input checks - duplicate data checks - unauthorized access checks - incorrect authentication checks - database or storage failure behavior - response validation The goal is to cover critical business rules, not just the happy path. For example, if a user submits empty email or password, the system should fail safely without creating broken data or an invalid session. Coverage should focus on: - registration logic - login logic - password validation - email verification - session management - reset/change password behavior - user retrieval from storage

7. Final Test Summary

Test Type	Total Tests	Passed	Failed
Unit Testing	30–40	To be recorded	To be recorded
Integration Testing	10–15	To be recorded	To be recorded
API Testing	15–20	To be recorded	To be recorded
Total	55–75	To be recorded	To be recorded

8. Defects Likely to Be Found

Automated tests would help identify defects such as: - duplicate email entries being accepted - password rules not enforced consistently - wrong current password accepted in some flows - verification code not cleared after successful verification - session restoration failing after page reload - invalid storage data not handled correctly - authentication logic failing after password updates - empty or malformed fields creating inconsistent user records These defects are common in validation-heavy systems and can be caught early with tests.

9. Conclusion

This project is a strong candidate for automated testing because it contains multiple validation rules and important user workflows. The most relevant testing approach is: - Unit testing for each function - Integration testing for complete user journeys - API testing for future backend integration Together, these three testing layers improve software reliability, reduce regression risk, and ensure the application remains stable after modifications. In conclusion, a structured automated testing framework would strengthen the project significantly and help maintain quality throughout future updates.